

Programación Python



Agenda

- Lectura de archivos
- Ciclos
- Entrada de Datos
- Funciones
- Ejemplos

Leyendo un archivo CSV

Para leer archivos csv vamos a usar una función del paquete pandas llamada **read_csv**, algunas de las opciones más importantes de esta función son:

sep/delimiter: Separador de columnas.

header: El número de la fila que contiene los nombres de columnas (en caso de existir nombres de columnas).

index_col: El número de fila que contiene los nombres de filas (en caso de existir nombres de filas).

dtype: Un diccionario que indica el tipo de dato de cada columna, en forma de {nombre_columna : tipo_dato}

na_values: El valor que indica los valores NA.

decimal: El separador de decimales.

encoding: La codificación del archivo.

```
import pandas as pd
datos_est = pd.read_csv('datos/ejemplo_estudiantes.csv', delimiter = ';', decimal = ",", header = 0, index_col = 0)
datos_est
```

	Matematicas	Ciencias	Espanol	Historia	EdFisica
Lucia	7.0	6.5	9.2	8.6	8.0
Pedro	7.5	9.4	7.3	7.0	7.0
Ines	7.6	9.2	8.0	8.0	7.5
Luis	5.0	6.5	6.5	7.0	9.0
Andres	6.0	6.0	7.8	8.9	7.3
Ana	7.8	9.6	7.7	8.0	6.5
Carlos	6.3	6.4	8.2	9.0	7.2
Jose	7.9	9.7	7.5	8.0	6.0
Sonia	6.0	6.0	6.5	5.5	8.7
Maria	6.8	7.2	8.7	9.0	7.0

Leyendo un archivo Excel

Pandas también nos brinda una manera de leer archivos excel, mediante la función **read_excel**. Algunos de sus opciones más útiles son:

sheet_name: El número o el nombre de la hoja que se quiere leer.

header: El número de la fila que contiene los nombres de columnas (en caso de existir nombres de columnas).

index_col El número de fila que contiene los nombres de filas (en caso de existir nombres de filas).

dtype: Un diccionario que indica el tipo de dato de cada columna, en forma de {nombre_columna : tipo_dato}

na_values: El valor que indica los valores NA.

```
datos_bic = pd.read_excel('datos/compra_bicicletas.xlsx', sheet_name = 0, header = 0, index_col = 0, engine = "openpyxl")
datos_bic
```

ID	Marital Status	Gender	Income	...	Region	Age	Purchased Bike
12496	Married	Female	40000	...	Europe	42	No
24107	Married	Male	30000	...	Europe	43	No
14177	Married	Male	80000	...	Europe	60	No
24381	Single	Male	70000	...	Pacific	41	Yes
25597	Single	Male	30000	...	Europe	36	Yes
...	...	...	...	...	...	..	...
23731	Married	Male	60000	...	North America	54	Yes
28672	Single	Male	70000	...	North America	35	Yes
11809	Married	Male	60000	...	North America	38	Yes
19664	Single	Male	100000	...	North America	38	No
12121	Single	Male	60000	...	North America	53	Yes

[1000 rows x 12 columns]

#Instalar en la terminal

pip install xlrd

pip install openpyxl

Ciclos

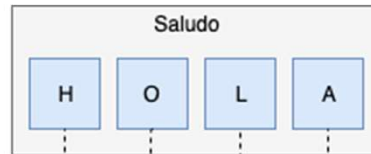
for letra **in** saludo:
 print(letra)

Primera iteración: letra = "H"

Segunda iteración: letra = "O"

Tercera iteración: letra = "L"

Cuarta iteración: letra = "A"



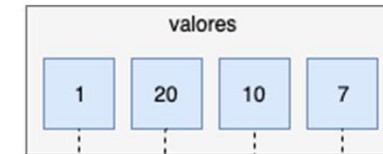
for elemento **in** valores:
 sumatoria = sumatoria + elemento

Primera iteración: sumatoria = 0 y elemento = 1

Segunda iteración: sumatoria = 1 y elemento = 20

Tercera iteración: sumatoria = 21 y elemento = 10

Cuarta iteración: sumatoria = 31 y elemento = 7



```
valores = [1, 20, 10, 7]
sumatoria = 0

for elemento in valores:
    print("sumatoria: %i\nelemento: %i" % (sumatoria, elemento))
    sumatoria = sumatoria + elemento
    print("sumatoria = sumatoria + elemento\nsumatoria: %i\n" % (sumatoria) + "="*15 )
```

Ciclos - for

No siempre nos conviene iterar en los elementos de una estructura iterable, en ocasiones nos sirve más iterar en los índices de esta estructura. Por ejemplo:

Si queremos imprimir la información de unos clientes, pero tenemos la información en dos listas distintas(**range**).

```
edades = [23, 20, 24]
nombres = ["Juan", "Pedro", "Maria"]
```

```
for indice in range(len(edades)):
    cadena = "Nombre: %s\nEdad:%i\n" % (nombres[indice], edades[indice])
    cadena = cadena + "=" * 15
    print(cadena)
```

```
for indice in range(len(edades)):
    cadena = "Nombre: %s\nEdad:%i\n" % (nombres[indice], edades[indice])
    cadena = cadena + "=" * 15
    print(cadena)
```

nombres		
0	1	2
"Juan"	"Pedro"	"Maria"

edades		
0	1	2
23	20	24

Primera iteración: indice = 0, nombres[0] = "Juan" y edades[0] = 23

Segunda iteración: indice = 1, nombres[1] = "Pedro" y edades[1] = 20

Tercera iteración: indice = 2, nombres[2] = "Maria" y edades[2] = 24

Ciclos en dataFrame

```
import pandas as pd
```

```
# Un diccionario
datos = {'nombre': ["Luis", "María", "Pepe"],
        'ciudad': ["San José", "Lima", "Buenos Aires"],
        'edad': [23, 56, 12]}

datos
```

```
{'nombre': ['Luis', 'María', 'Pepe'], 'ciudad': ['San José', 'Lima', 'Buenos Aires'], 'edad': [23, 56, 12]}
```

```
mi_df = pd.DataFrame(datos)
mi_df
```

	nombre	ciudad	edad
0	Luis	San José	23
1	María	Lima	56
2	Pepe	Buenos Aires	12

```
tam_filas, tam_columnas = mi_df.shape

for indice_fila in range(tam_filas):
    for indice_columna in range(tam_columnas):
        cadena = "valor en mi_variable[%i,%i] = %s"
        cadena = cadena % (indice_fila, indice_columna, mi_df.iloc[indice_fila, indice_columna])
        print(cadena)
```

```
for nombre_fila in mi_df.index:
    for nombre_columna in mi_df.columns:
        cadena = "valor en mi_variable['%s','%s'] = %s"
        cadena = cadena % (nombre_fila, nombre_columna, mi_df.loc[nombre_fila, nombre_columna])
        print(cadena)
```

iloc lo usamos para acceder a los valores mediante los índices

loc lo usamos para acceder a los valores por los nombres

Ciclos - While

Cuando usamos ciclos podemos usar **break** para parar el ciclo y continuar el flujo de la ejecución.

```
contador = 1
while contador <= 15:
    print(contador)
    if contador == 10:
        break
    contador = contador + 1
```

```
1
2
3
4
5
6
7
8
9
10
```

```
print("último valor de contador: %i" % contador)
```

```
último valor de contador: 10
```

También podemos usar **continue** para brincar a la siguiente iteración del ciclo. Es especialmente útil cuando queremos omitir el cálculo de algún valor por algún motivo.

```
contador = 0
while contador <= 15:
    contador = contador + 1
    if contador % 2 == 0:
        continue
    print(contador)
```

```
1
3
5
7
9
11
13
15
```

```
print("último valor de contador: %i" % contador)
```

```
último valor de contador: 16
```


Entrada de Datos y while

En Python para ingresar datos desde el teclado ocupamos usar input. Ejemplo de un programa que lee un número y nos diga si es múltiplo de 2:

```
lectura = "\nDigite un número:"
lectura = lectura + "\n(Digite 'salir' para terminar el programa) "

mensaje = ""

while mensaje != 'salir':
    mensaje = input(lectura)
    if not mensaje.isnumeric():
        continue
    numero = int(mensaje)
    if numero % 2 == 0:
        print("\nEl número " + str(numero) + " es par.")
    else:
        print("\nEl número " + str(numero) + " es impar.")
```

```
lectura = "\nDigite un número:"
lectura = lectura + "\n(Digite 'salir' para terminar el programa) "

while True: # Muy peligroso
    mensaje = input(lectura)
    if mensaje == "salir":
        break
    if not mensaje.isnumeric():
        continue
    numero = int(mensaje)
    if numero % 2 == 0:
        print("\nEl número " + str(numero) + " es par.")
    else:
        print("\nEl número " + str(numero) + " es impar.")
```

Funciones

Para hacer una función debemos usar **def** de la siguiente manera

```
def nombre_funcion(< parametro_1, parametro_2, ... >):  
    < cuerpo de la función >
```

```
def sumar(x, y):  
    z = x + y  
    return z
```

```
sumar(5, 6)
```

```
11
```

Nota: Los parámetros son opcionales y podemos poner cuantos queramos.

Retornando **Múltiples Valores**: tupla, lista o diccionario

```
def cuadrado_valores(lado):  
    respuesta = {}  
    respuesta["area"] = lado ** 2  
    respuesta["perimetro"] = lado*4  
    return respuesta
```

```
cuadrado_valores(5)
```

```
{'area': 25, 'perimetro': 20}
```

```
cuadrado_valores(16)
```

```
{'area': 256, 'perimetro': 64}
```

La siguiente función recibe un DataFrame (DF) y retorna en un diccionario el valor mínimo, el máximo, la cantidad de ceros y la cantidad de números pares que contiene el dataFrame:

```
def valores(DF):
    n, m = DF.shape
    minimo = DF.iloc[0,0]
    maximo = minimo
    total_ceros = 0
    total_pares = 0
    for i in range(n):
        for j in range(m):
            if DF.iloc[i,j] > maximo:
                maximo = DF.iloc[i,j]
            if DF.iloc[i,j] < minimo:
                minimo = DF.iloc[i,j]
            if DF.iloc[i,j] == 0:
                total_ceros = total_ceros+1
            if DF.iloc[i,j] % 2 == 0:
                total_pares = total_pares+1

    respuesta = {'Maximo' : maximo, 'Minimo' : minimo,
                'Total_Ceros' : total_ceros, 'Pares' : total_pares}
    return respuesta
```

La siguiente función recibe un DataFrame (DF) y un número de columna (nc) y retorna en un diccionario el nombre de la variable correspondiente al número de columna, la media, la mediana, la desviación estándar, la varianza, el máximo y el mínimo:

```
import numpy as np

def estadisticas(DF,nc):
    media = np.mean(DF.iloc[:,nc])
    mediana = np.median(DF.iloc[:,nc])
    desviacion = np.std(DF.iloc[:,nc])
    varianza = np.var(DF.iloc[:,nc])
    maximo = np.max(DF.iloc[:,nc])
    minimo = np.min(DF.iloc[:,nc])
    return {'Variable' : DF.columns.values[nc],
            'Media' : media,
            'Mediana' : mediana,
            'DesEst' : desviacion,
            'Varianza' : varianza,
            'Maximo' : maximo,
            'Minimo' : minimo}
```

```
estadisticas(datos_est, 1)
```

```
{'Variable': 'Ciencias', 'Media': 7.650000000000001, 'Mediana': 6.85, 'DesEst': 1.5272524349301262, 'Varianza': 2.3324999999999999, 'Maximo': 9.7, 'Minimo': 6.0}
```

Ejemplos

1. Obtenga una lista con 20 números aleatorios entre 10 y 80

```
import numpy as np

np.random.randint(10, 80, 20)
```

```
array([70, 43, 65, 16, 13, 43, 12, 72, 16, 20, 31, 44, 18, 13, 64, 70, 35,
      11, 61, 19])
```

2. Si tenemos un diccionario con edades de clientes que quieren ver una obra de teatro y queremos agregar una nueva llave llamada "Precio". La entrada cuesta 5000 colones para adultos (18 años en adelante), 4000 colones para personas entre los 15 y 18 años, y para las personas de 15 años o menores se cobra 3000 colones.

```
clientes = {"Num Cliente": [1, 2, 3, 4, 5, 6, 7],
            "Edad": [23, 18, 28, 12, 33, 21, 17]}

precio = []

for edad in clientes["Edad"]:
    if edad >= 18:
        precio.append(5000)
    if 15 < edad < 18:
        precio.append(4000)
    if edad <= 15:
        precio.append(3000)

clientes["Precio"] = precio
clientes
```

```
{'Num Cliente': [1, 2, 3, 4, 5, 6, 7], 'Edad': [23, 18, 28, 12, 33, 21, 17], 'Precio': [5000, 5000, 5000, 3000, 5000, 5000, 4000]}
```

3. Si tenemos unos usuarios de un sistema y queremos obtener un diccionario con los nombres completos, pero si los nombres completos tienen más de 14 caracteres entonces solo la inicial del nombre más el apellido.

```
usuarios = {  
    'ma_cespedes': {  
        'nombre': 'maría',  
        'apellido': 'céspedes',  
        'lugar': 'alajuela',  
    },  
    'ma_alfaro': {  
        'nombre': 'mario',  
        'apellido': 'alfaro',  
        'lugar': 'heredia',  
    },  
    'ju_rojas': {  
        'nombre': 'julián',  
        'apellido': 'rojas',  
        'lugar': 'curridabat',  
    },  
}
```

```
nombres_completos = {}  
  
for usuario, info_usuario in usuarios.items():  
    nombre = info_usuario['nombre'] + " " + info_usuario['apellido']  
    if len(nombre) >= 14:  
        nombre = info_usuario['nombre'][0] + " " + info_usuario['apellido']  
  
    nombres_completos[usuario] = nombre.title()  
  
nombres_completos
```

```
{'ma_cespedes': 'M Céspedes', 'ma_alfaro': 'Mario Alfaro', 'ju_rojas': 'Julián Rojas'}
```

4. Defina una función que reciba un vector de números y retorne en una lista la lista de valores pares y otra con la lista de valores impares

```
def pares_e_impares(l):  
    pares = []  
    impares = []  
    for x in l:  
        if x % 2 == 0:  
            pares = pares + [x]  
        else:  
            impares = impares + [x]  
    return [pares, impares]
```

```
pares_e_impares([9,3,7,6,4,5,6,9,8,1,2,3,5,4,7])
```

```
[[6, 4, 6, 8, 2, 4], [9, 3, 7, 5, 9, 1, 3, 5, 7]]
```

5. Cargue la tabla de ejemplo de estudiantes y desarrolle una función que retorne un diccionario con el nombre, la materia y la nota del estudiante que obtuvo la mayor nota en cualquiera de las materias.

```
datos_estudiantes = pd.read_csv('datos/ejemplo_estudiantes.csv', delimiter = ';', decimal = ",", index_col=0)

def mayor_nota(df):
    filas = df.shape[0]
    cols = df.shape[1]
    max_nota = 0
    for i in range(filas):
        for j in range(cols):
            if df.iloc[i, j] > max_nota:
                max_nota = df.iloc[i, j]
                res = {'Nombre' : datos_estudiantes.index[i], 'Materia' : df.columns[j], 'Nota' : max_nota}
    return res
```

```
mayor_nota(datos_estudiantes)
```

```
{'Nombre': 'Jose', 'Materia': 'Ciencias', 'Nota': 9.7}
```


Practica

1. Cargue la tabla de datos que esta en el archivo diabetes v2.csv haga lo siguiente:
 - Calcule la dimensión de la Tabla de Datos.
 - Calcule la suma de las columnas con variables cuantitativas (numéricas).
 - Calcule la moda de las columnas con variables cualitativas (categóricas).
2. Usando for(...) en Python muestre los números del 1 al 100 que terminan en 3.
3. Mediante un ciclo, calcule la sumatoria de los números enteros multiplos de 13, comprendidos entre el 1 y el 200.
4. Mediante un ciclo, guarde en una lista todos los números impares desde 1 hasta el 15.
5. Programe en Python una función que recibe tres valores A, B, y C y retorna menor.
6. Programe en Python una función que genera 200 numeros al azar entre 1 y 500 y luego calcula cuantos están entre el 50 y 450, ambos inclusive.
7. Desarrolle una funcion que reciba dos numeros enteros a y b, tal que, en un diccionario retorne el Máximo Común Divisor (MCD) y el minimo comun multiplo (mcm). Para calcular el MCD puede utilizar la funcion **gcd** del paquete **math**. La formula para calcular el mcm es la siguiente:
$$\text{mcm}(a, b) = \frac{a \cdot b}{\text{MCD}(a, b)}$$

8. Desarrolle una función que recibe una matriz cuadrada A de tamaño $n \times n$ y retorna su transpuesta (utilizando for). Por ejemplo, la transpuesta de la matriz A:

$$A = \begin{pmatrix} 0 & 20 & 6 \\ 4 & 12 & 48 \\ -8 & -5 & -16 \end{pmatrix}$$

es la siguiente matriz A^t :

$$A^t = \begin{pmatrix} 0 & 4 & -8 \\ 20 & 12 & -5 \\ 6 & 48 & -16 \end{pmatrix}$$

9. Desarrolle una función que recibe un **DataFrame** y un nombre de columna y que retorne un diccionario con los valores de la media, mínimo, máximo y mediana de dicha columna del dataframe. Verifique la correctitud de esta función usando la tabla `Players1.csv` y la columna `minutes`, que hace referencia a los minutos de juego.

10. Desarrolle una función que recibe un **DataFrame** y dos números de columna y que retorna la correlación entre esas dos variables. Verifique la correctitud de esta función usando la tabla `Players1.csv` y las columnas 5 y 6, que hacen referencia a las variables **shots** y **passes**.